Embedded System Architecture - CSEN 701

Module 4: *ES Modeling & Design*

Lecture 09: *System Modeling using StateCharts*

Dr. Eng. Catherine M. Elias

catherine.elias@guc.edu.eg

Lecturer, Computer Science and Engineering,
Faculty of Media Engineering and Technology, German University in Cairo

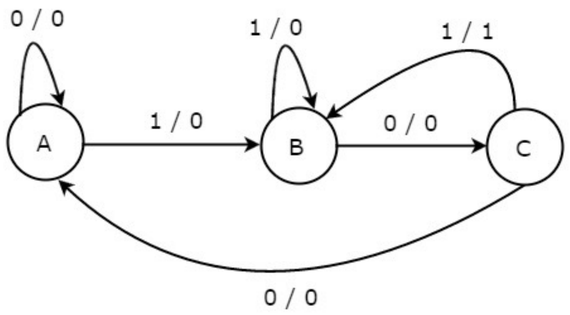
Outline

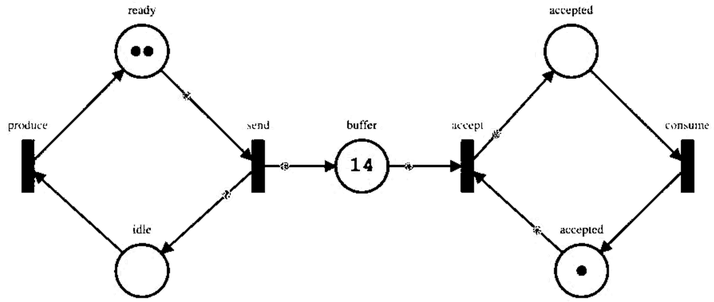
- Event-driven Discrete Model
- StateCharts
- Examples

Event-driven Discrete Model

Techniques

- Finite State Automata / Machine (FSA/FSM)
- Petri-Nets
- Statecharts





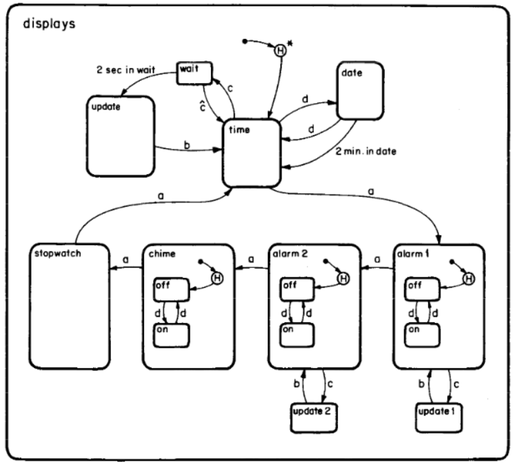


Fig. 13.

Statecharts

Definition

- Statecharts are a visual formalism for modeling the behavior of systems, particularly **reactive systems**.
- They were introduced by David Harel in the 1980s as an **extension of finite state machines** (FSMs) to better handle the complexity of software systems.
- Statecharts provide a way to represent the behavior of a system in a **hierarchical and modular fashion**.

Definition

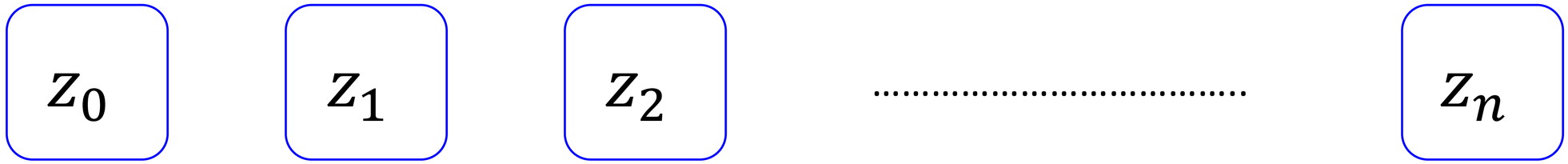
$$Statecharts = Mealy + Moore + Extra\ Concepts$$

Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The States:

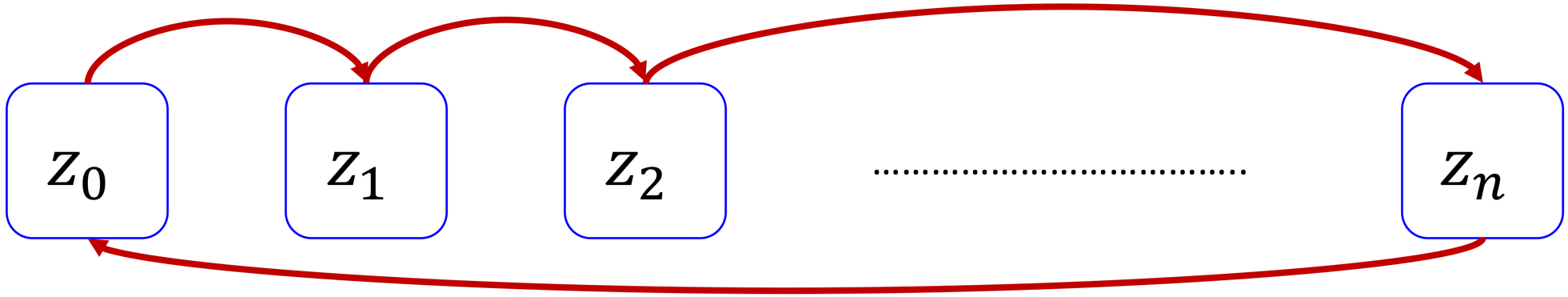
- They represent the different modes or conditions that a system can be in.
- *It is important to note that the state is represented by a Rounded Box in the graphical SC model*



Statecharts = Mealy + Moore + Extra Concepts

The Transition:

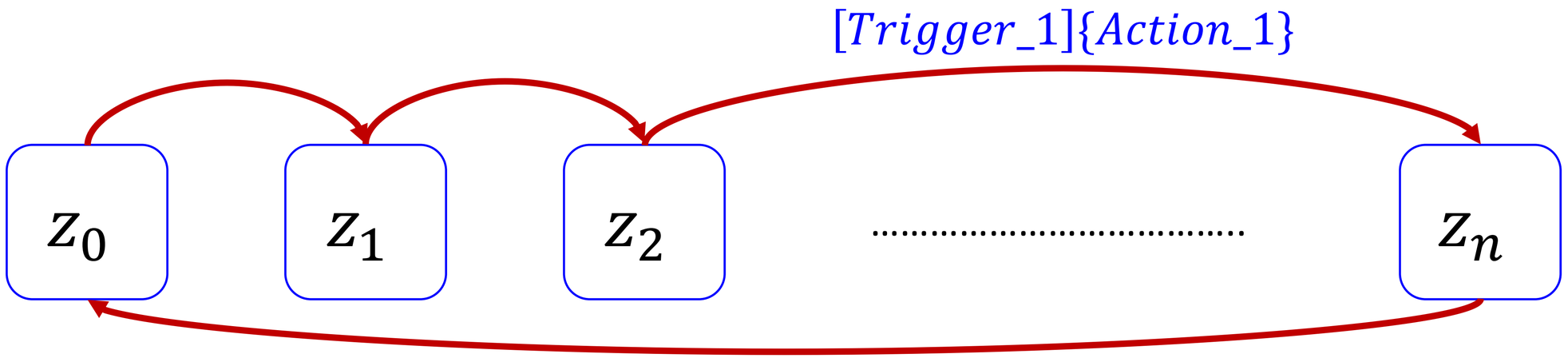
- Transitions describe the conditions under which a system moves from one state to another.
- Transitions are triggered by events
- *It is important to note that the transition is represented by an Arrow in the graphical SC model*



Statecharts = Mealy + Moore + Extra Concepts

The Transition:

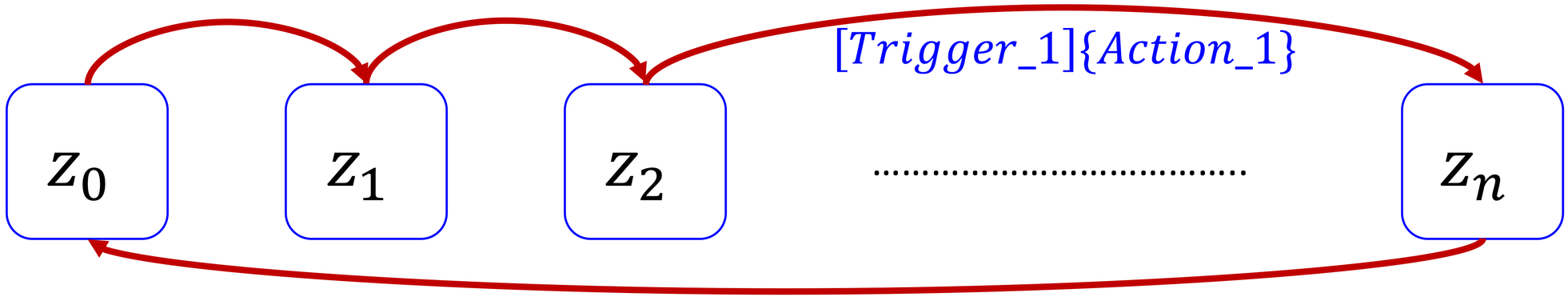
- The transition means that the states are executed ***sequentially***.
- The transitions carry an event that triggers the system to change its mode of operation from one state to another.
- The transition should carry the trigger/action expression



Statecharts = Mealy + Moore + Extra Concepts

The input/trigger/event/guard Representation:

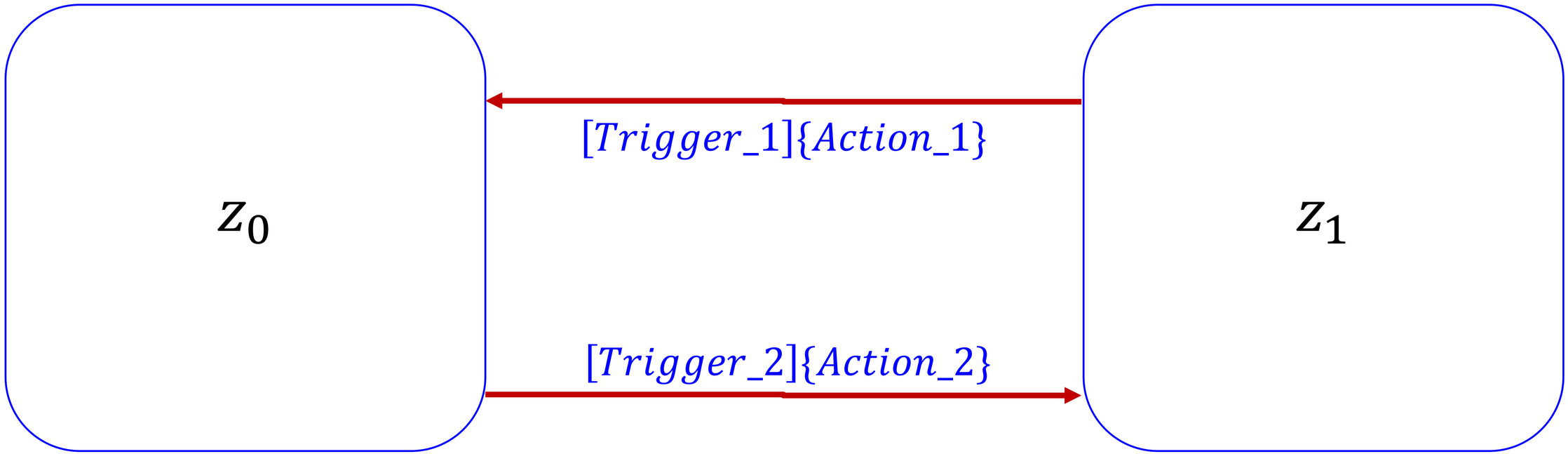
- The input can take any of the following forms:
  - Binary Event {0,1}, Ex.: $[u_1 = 1]$
  - Guard/condition, Ex.: $[Temperature < 25°]$
  - A state is active using **In(Z_0)**
  - Events can be generated when a certain condition is satisfied using **when(condition)**
  - Events can be generated after a state is activated by a defined time interval using **after(time)**



Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The States:

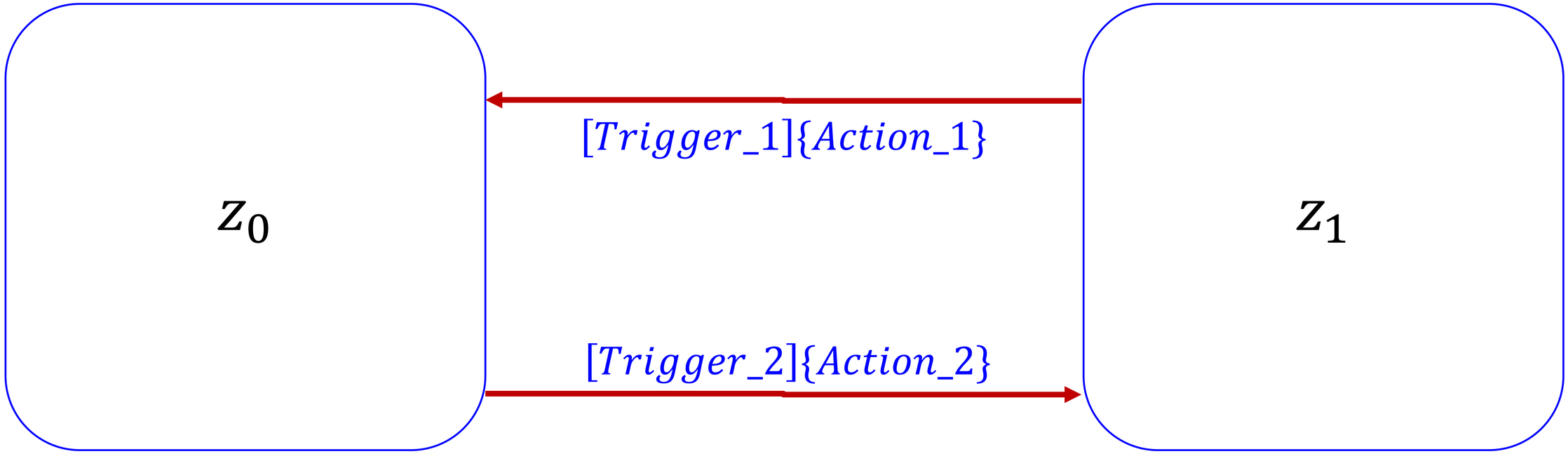


*This would make our SC like a **Mealy FSM***
(Action on the transition)

Statecharts = Mealy + Moore + Extra Concepts

The States:

- The states in Statecharts can either be:
  - **Simple (as in FSM)** or
  - **Composite/Super (Extra concept introduced with the SC)**.



Statecharts

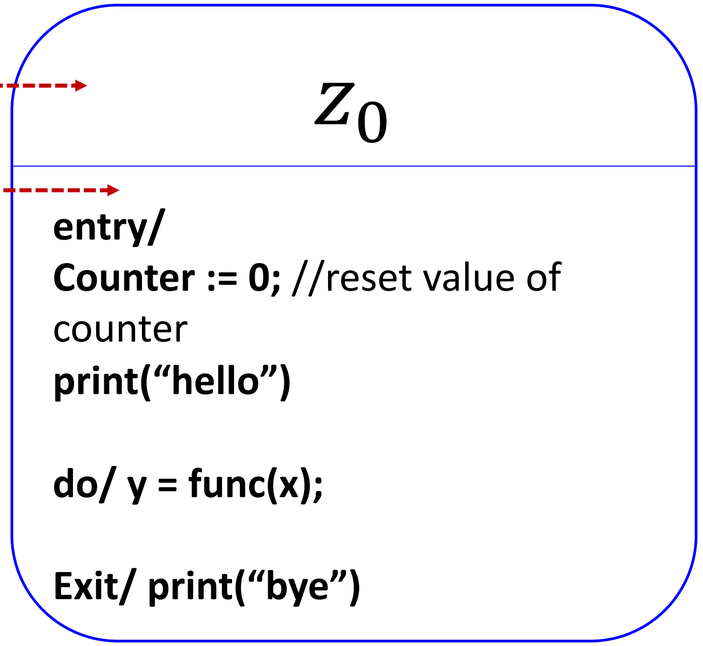
Statecharts = Mealy + Moore + Extra Concepts

The *Simple* States:

- The simple state is composed of two compartments.
- The first compartment contains the state name
- The second compartment is an internal activities compartment.
- This compartment holds a list of internal behaviors/activities that occurs inside the state
- These activities take the following format:

$$< behavior - type - label > / < behavior - Expression >$$

- [entry, exit, do] are the internal activities labels.
- The behavior Expression can basically be any method/function.



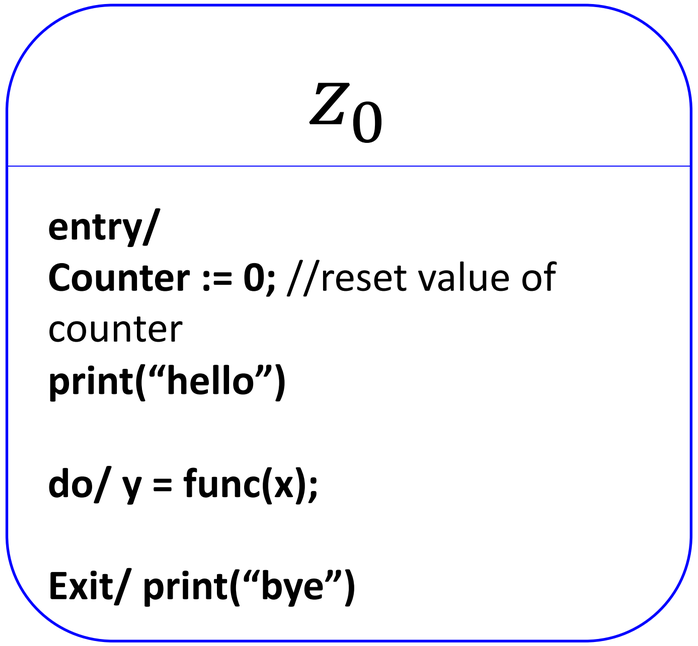
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The *Simple* States:

- **entry:** a behavior/action that will be executed upon entry to the state.
- **exit:** a behavior/action that will be executed upon exit from the state.
- **do:** a behavior/action that will be executed as long as the object is in the state or until the computation specified by the expression is completed. This represents ongoing behavior. The **do** behavior commences execution when the state is entered **only after** the state **entry** behavior has **completed**.

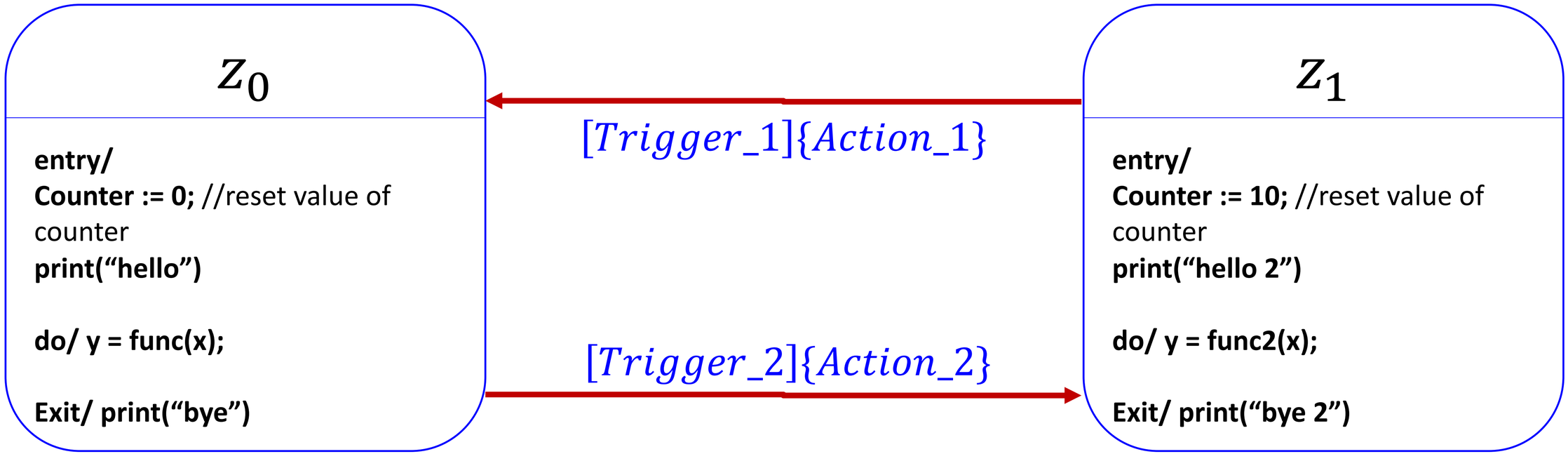
*This would make our SC like a **Moore FSM***
(Action inside the state)



Statecharts

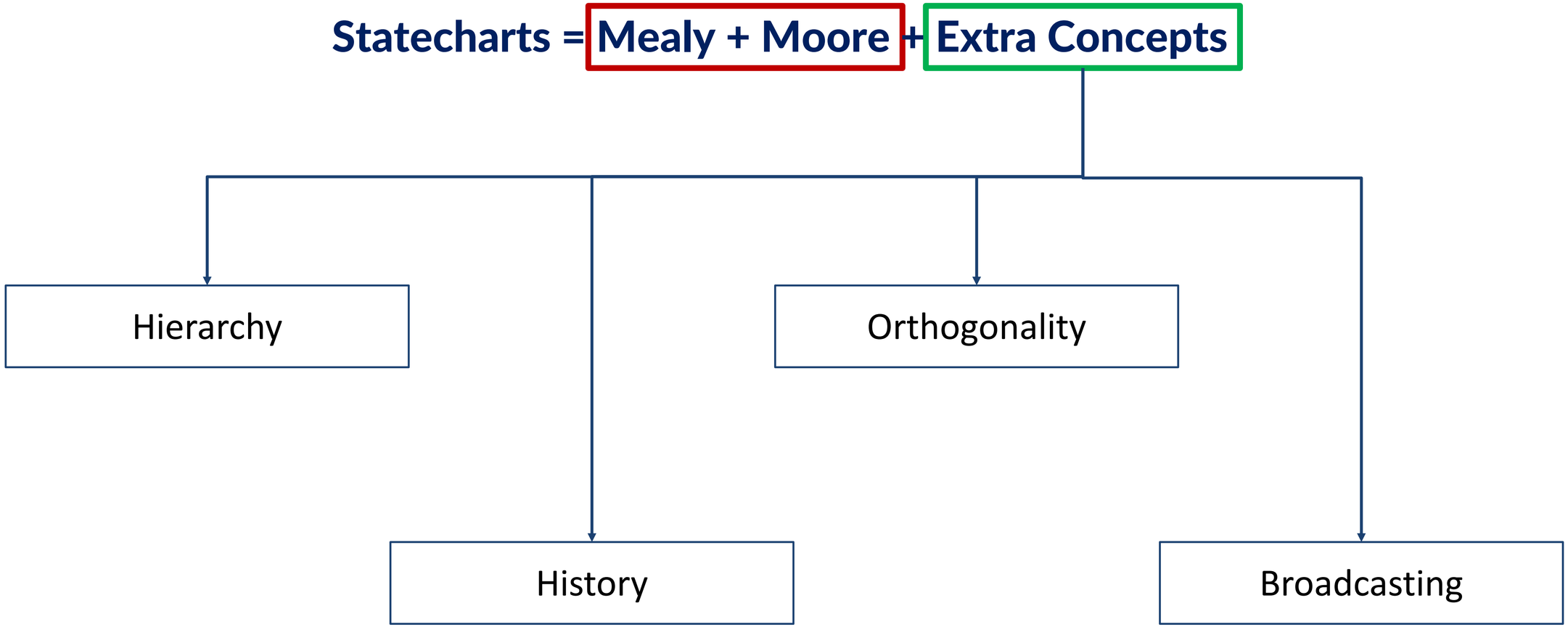
Statecharts = Mealy + Moore + Extra Concepts

The *Simple* States:



*This would make our SC like a **Mealy + Moore FSM***
(Action on the transition and inside the state)

Statecharts



Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- Statecharts support hierarchical structuring of states, allowing for a more modular and organized representation of complex systems.
- This hierarchy makes it easier to manage the complexity of large systems by breaking them down into smaller, more manageable components.

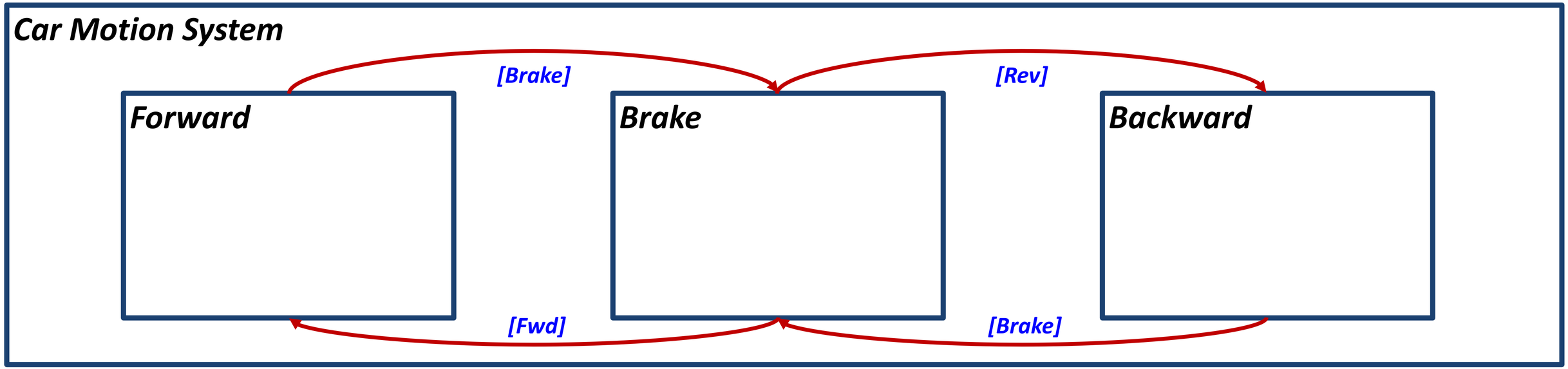
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- With the hierarchal concept of SC, a new type of state has been introduced known as superstates (Composite) and substates (Simple).
- The superstates are used to group a number of substates states.

Example:

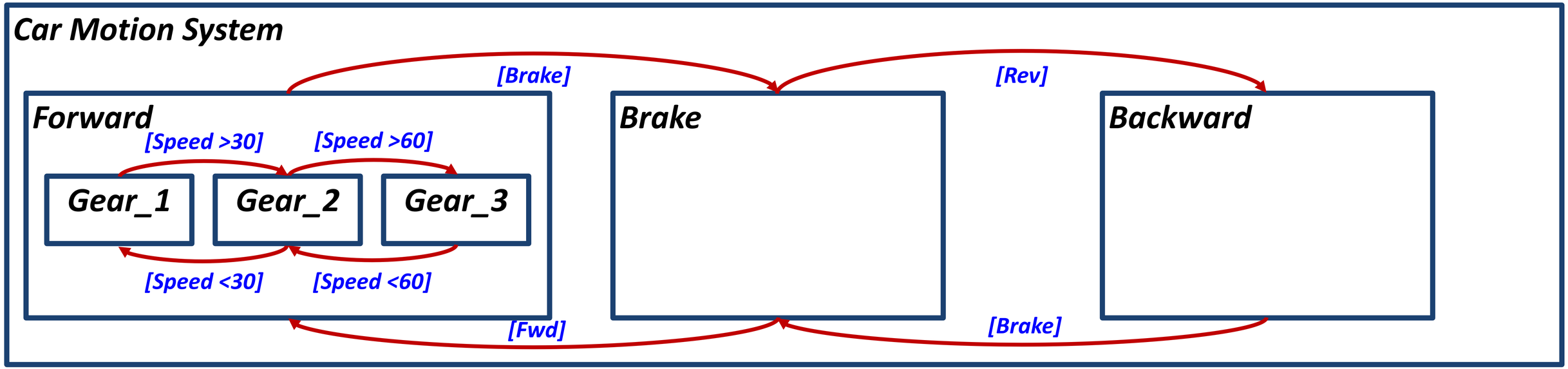


Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

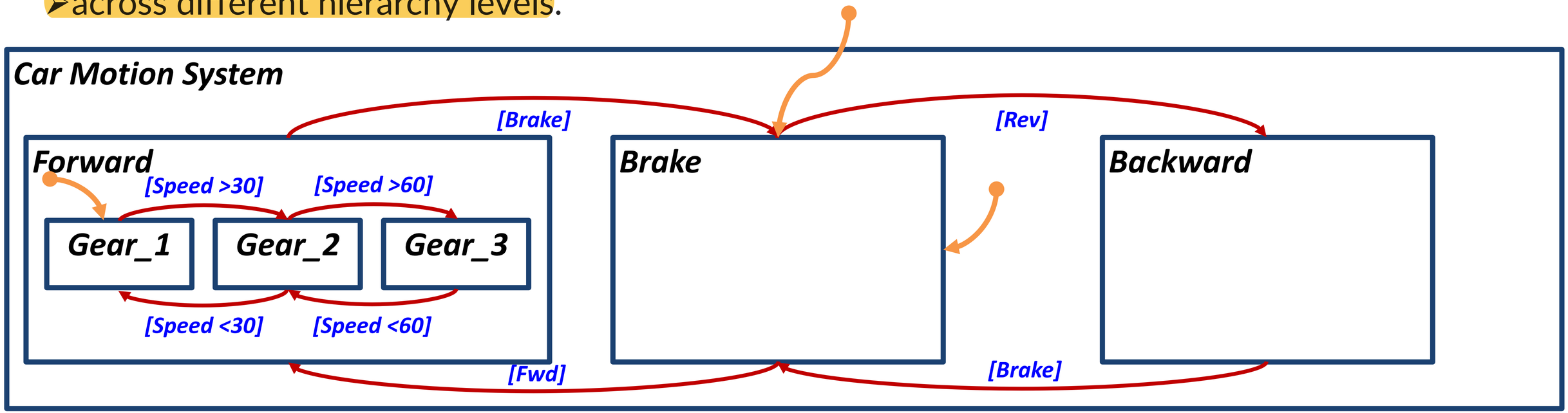
- Each state can be extended into a number of substates, introducing more details in the system model.
- Example:



Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy: Initial State

- There only exists one initial state for Mealy and Moore machines.
- Statecharts can define an initial state:
  - on different hierarchy levels and
  - across different hierarchy levels.

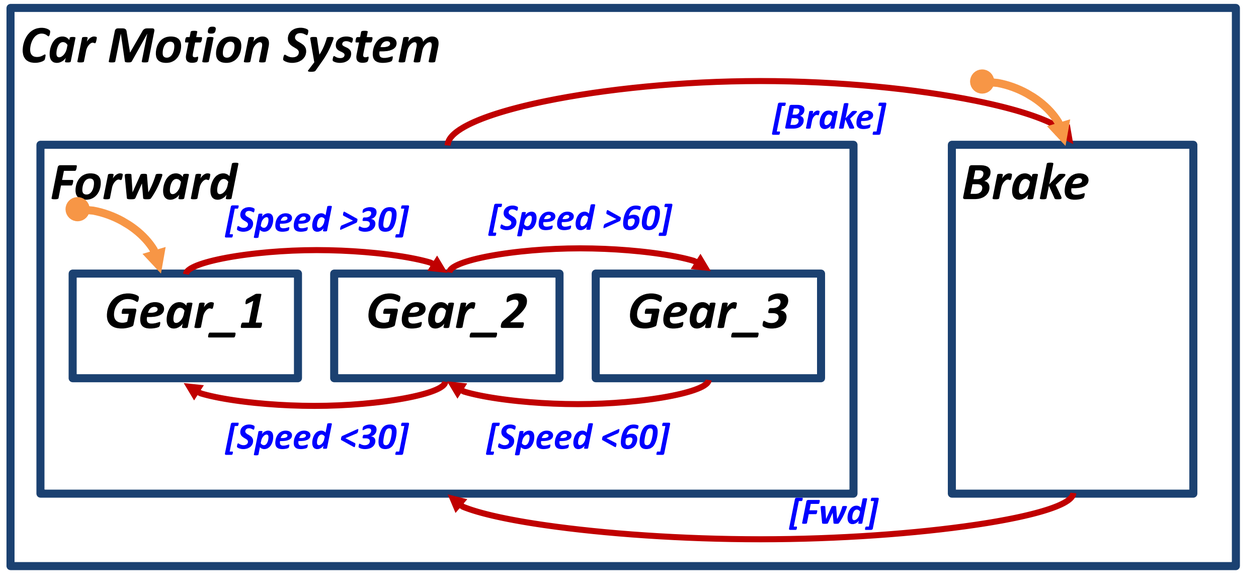


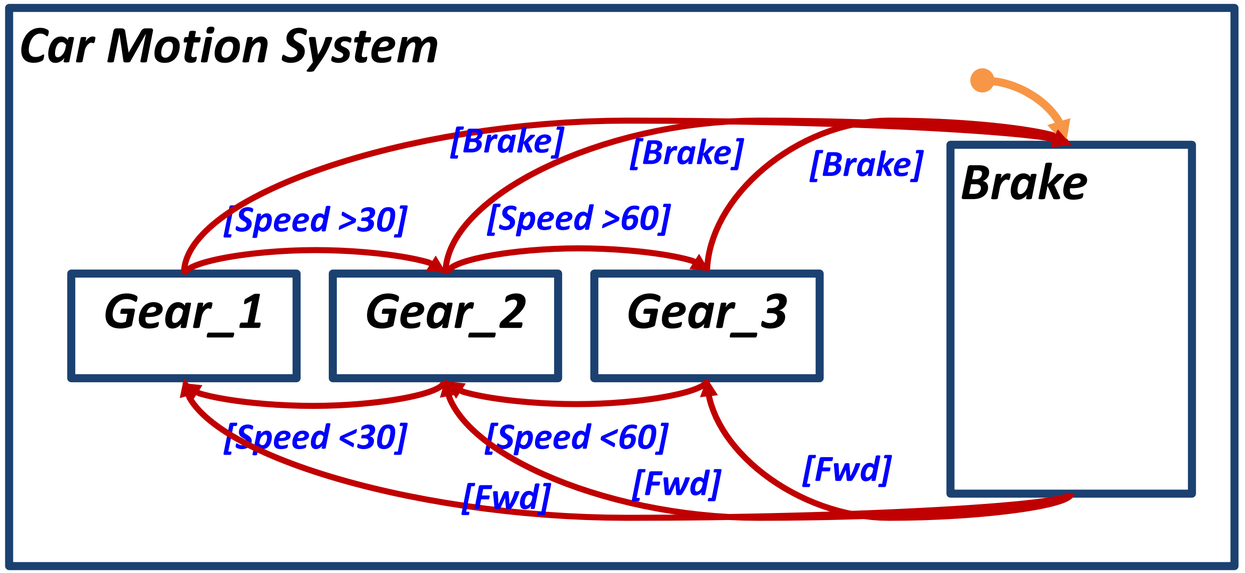
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- The hierarchy concept allows the designer to have an **abstract overview** of the system model. As well as a detailed view of each functionality
- The hierarchy concept helps in **decreasing** the number of defined transitions.
- Example:



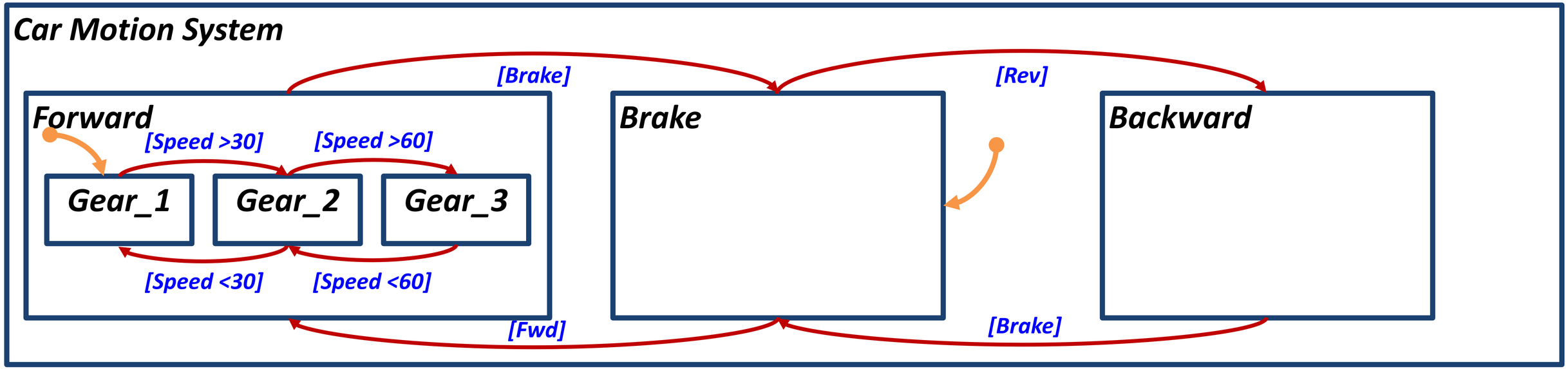


Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- When a superstate is active. One of its substates must be active.
- Example: Active states (Motion/ Forward/ Gear_1)

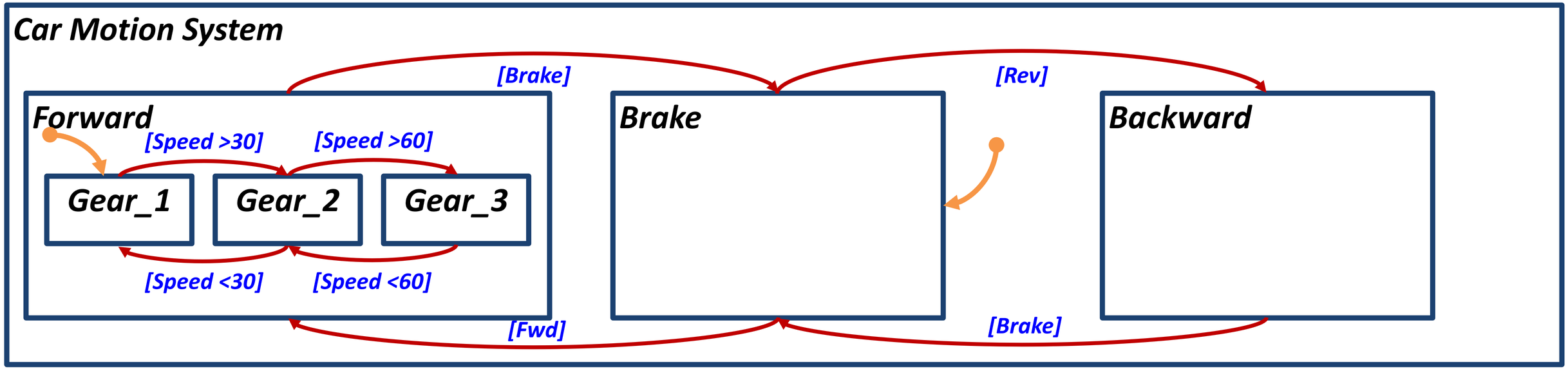


Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- Transitions can be defined moving to a superstate or a substate
- In case a transition's destination is a superstate. The defined initial substate is activated.
- Example:

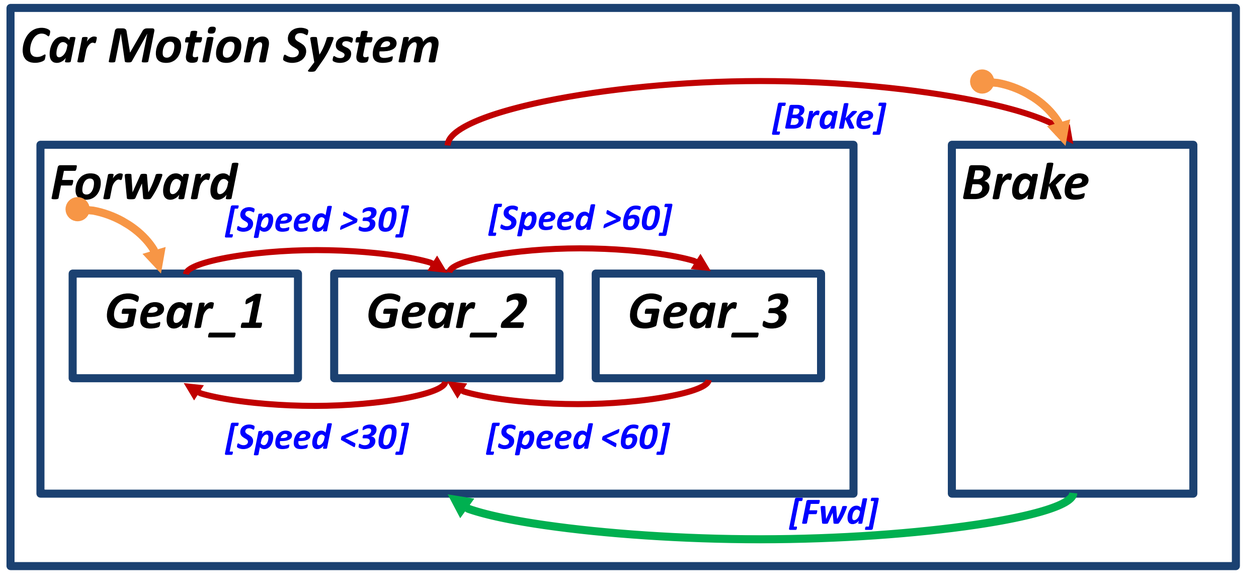


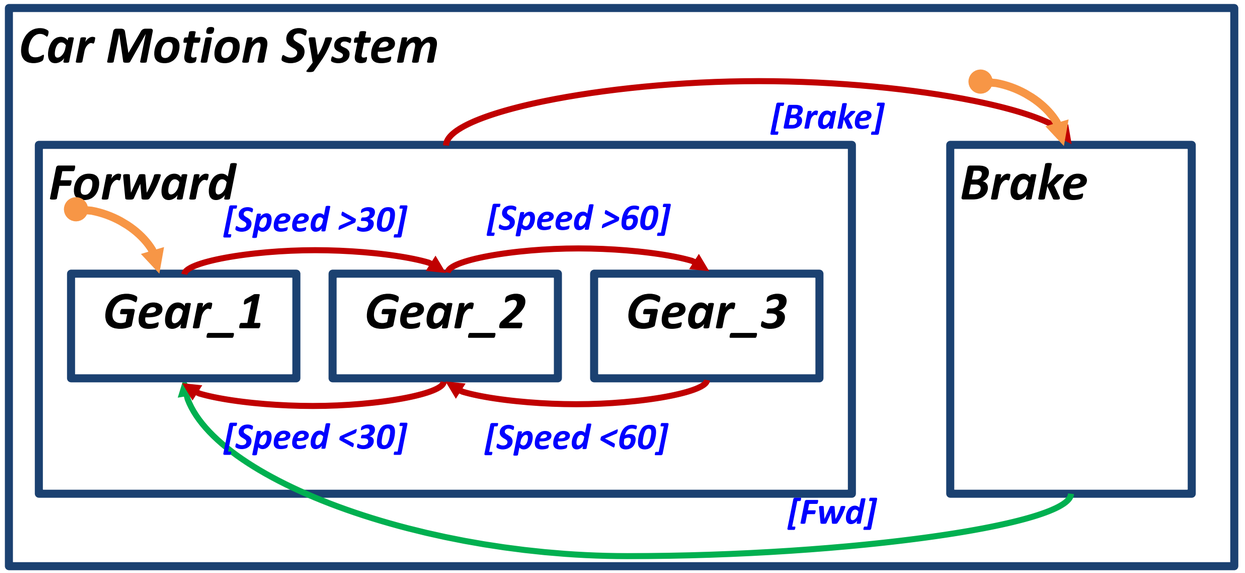
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

- Transitions can be defined moving to a superstate or a substate
- In case a transition's destination is a superstate. The defined initial substate is activated.
- Example: In both diagrams, Gear 1 will be activated after the transition from the brake



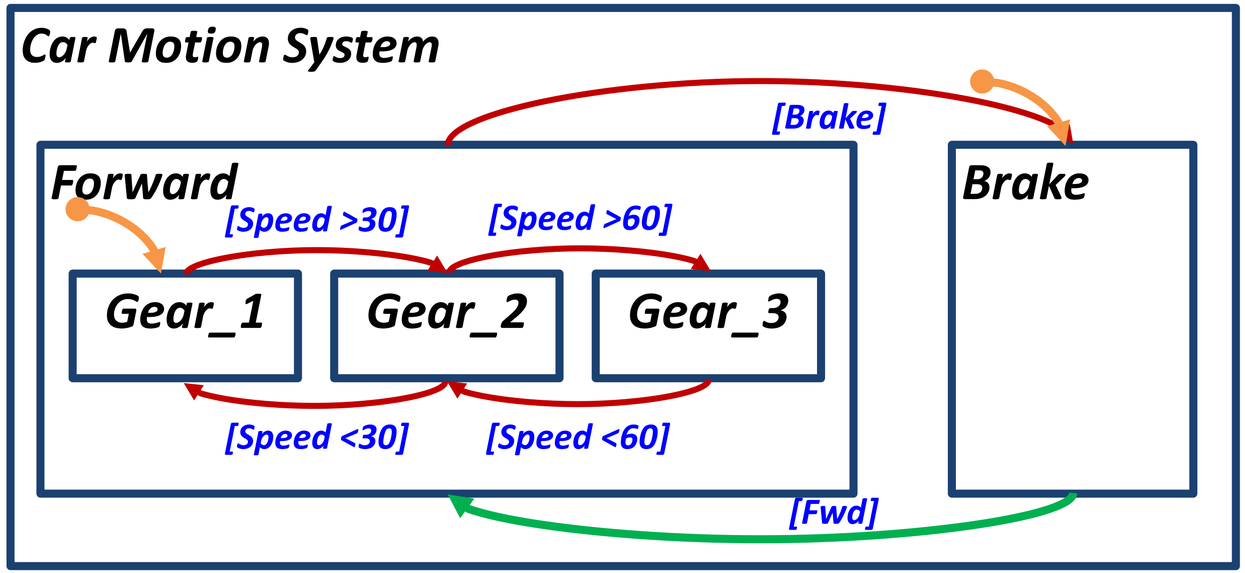


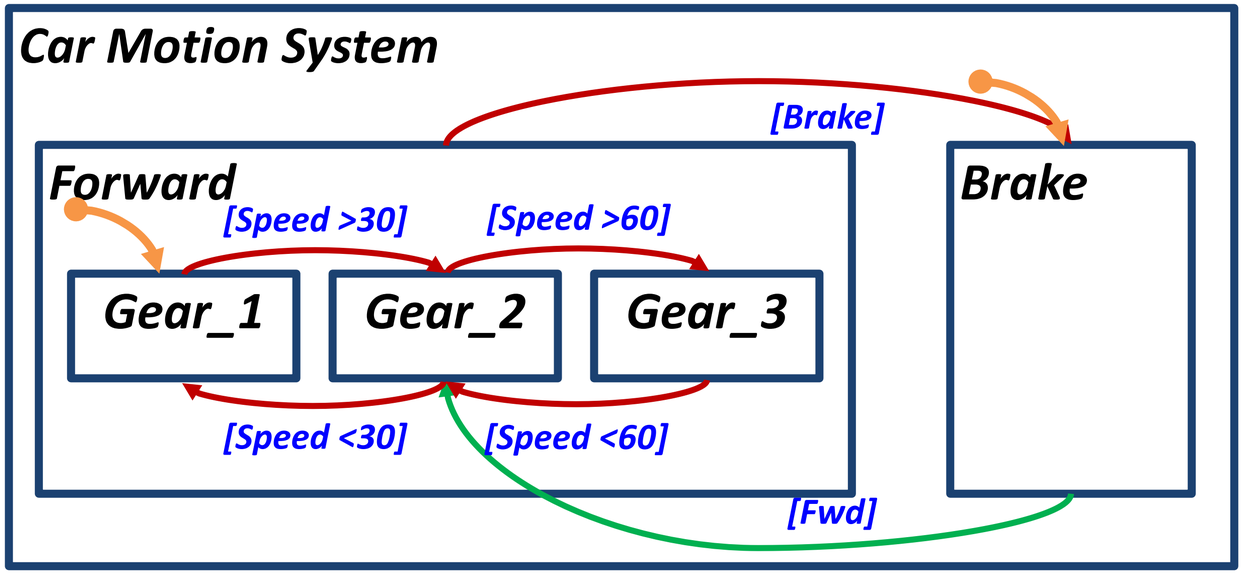
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Hierarchy:

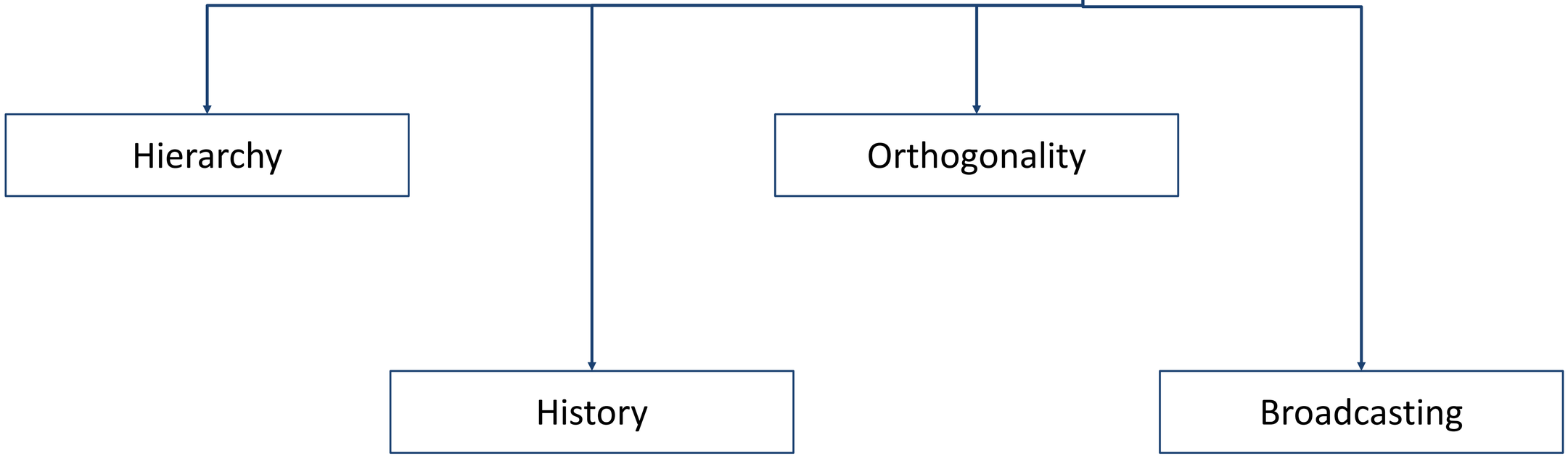
- Transitions can be defined moving to a superstate or a substate
- In case a transition's destination is a superstate. The defined initial substate is activated.
- Example: In diagram 1, Gear 1 will be activated, while in diagram 2, Gear 2 will be activated





Statecharts

Statecharts = Mealy + Moore + Extra Concepts



Statecharts

Statecharts = Mealy + Moore + Extra Concepts

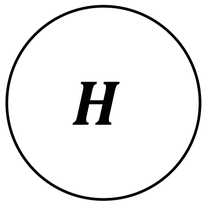
The History:

- The history concept in Statecharts refers to the ability to capture and represent the evolution of a system over time.
- Unlike traditional finite state machines (FSMs), Statecharts introduced the idea of a history mechanism to preserve the past states of a system, allowing for a more nuanced representation of system behavior.
- The history concept in Statecharts addresses a limitation of FSMs, where once a state is left, the system loses information about its previous state.
- In many real-world scenarios, retaining knowledge of the system's history is crucial for understanding and modeling its behavior accurately.

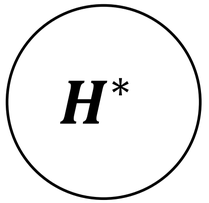
Statecharts = Mealy + Moore + Extra Concepts

The History:

- There are two main types of history concepts in Statecharts:
  - **Shallow History:**
    - Shallow history, represented by a small circle with H, indicates that when a state with shallow history is re-entered, the system resumes from the last substate it was in when the state was exited.
    - Shallow history captures the immediate past state **within a specific hierarchical level**.



  - **Deep History:**
    - Deep history, represented by a small circle with H^*, goes a step further.
    - It not only remembers the immediate past substate within a state but also remembers **the past substates of all nested states**.
    - When a state with deep history is re-entered, the system resumes from the exact configuration it was in when the state was exited.
    - Deep history provides a more comprehensive view of the system's past states.



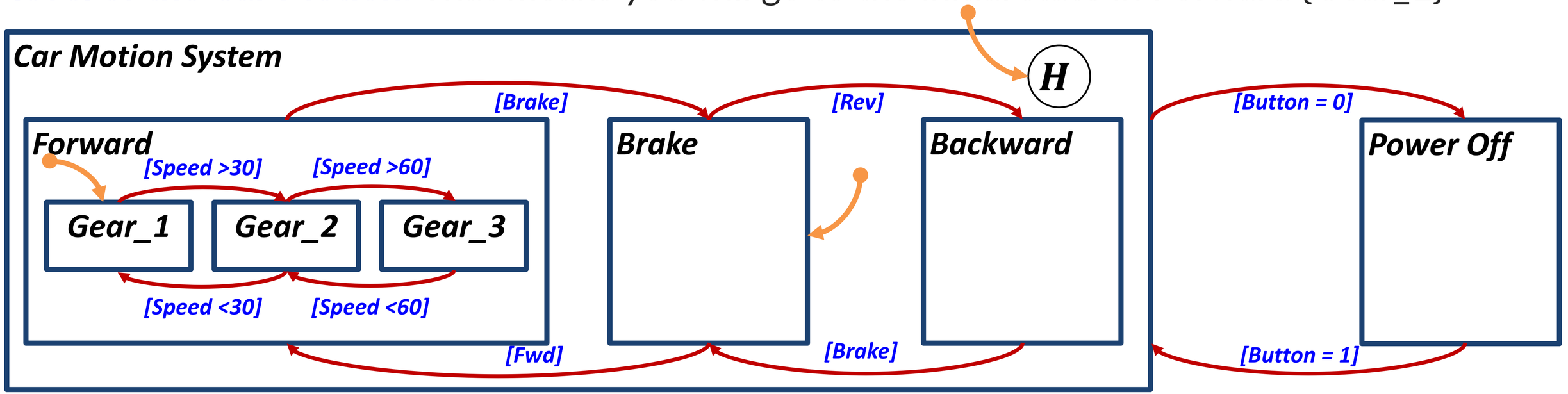
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The History:

Example: Let's assume that the motion system is connected sequentially with power off state and we are adding a **shallow history**. Assuming last active was Gear_3 on Forward.

This means that when you power off then back on, you will go to **the last active** state in the **level of the Car Motion.** That means you will go to the initial state in Forward {Gear_1}



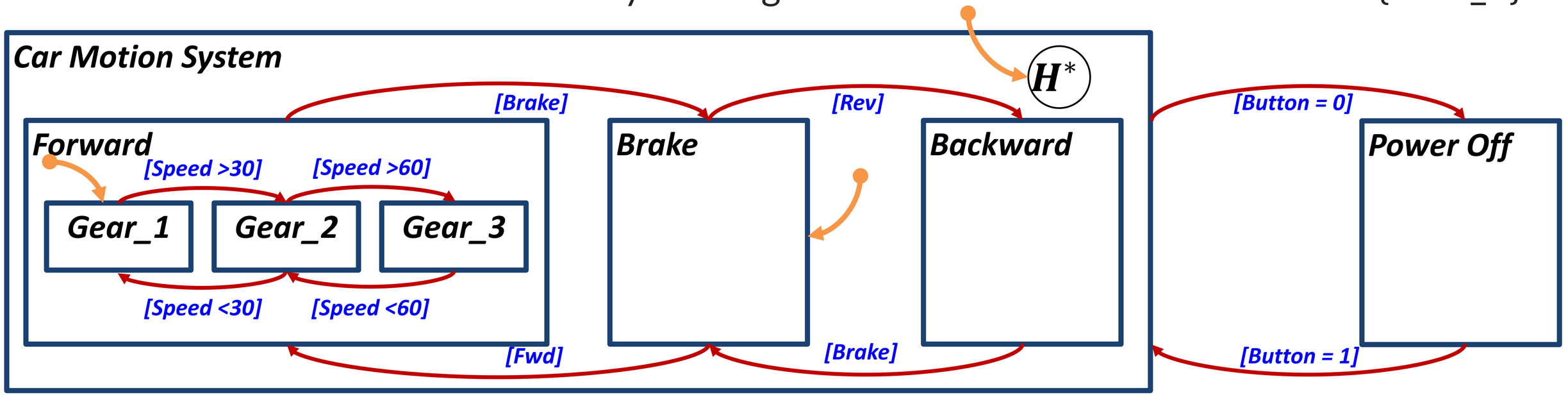
Statecharts

Statecharts = Mealy + Moore + Extra Concepts

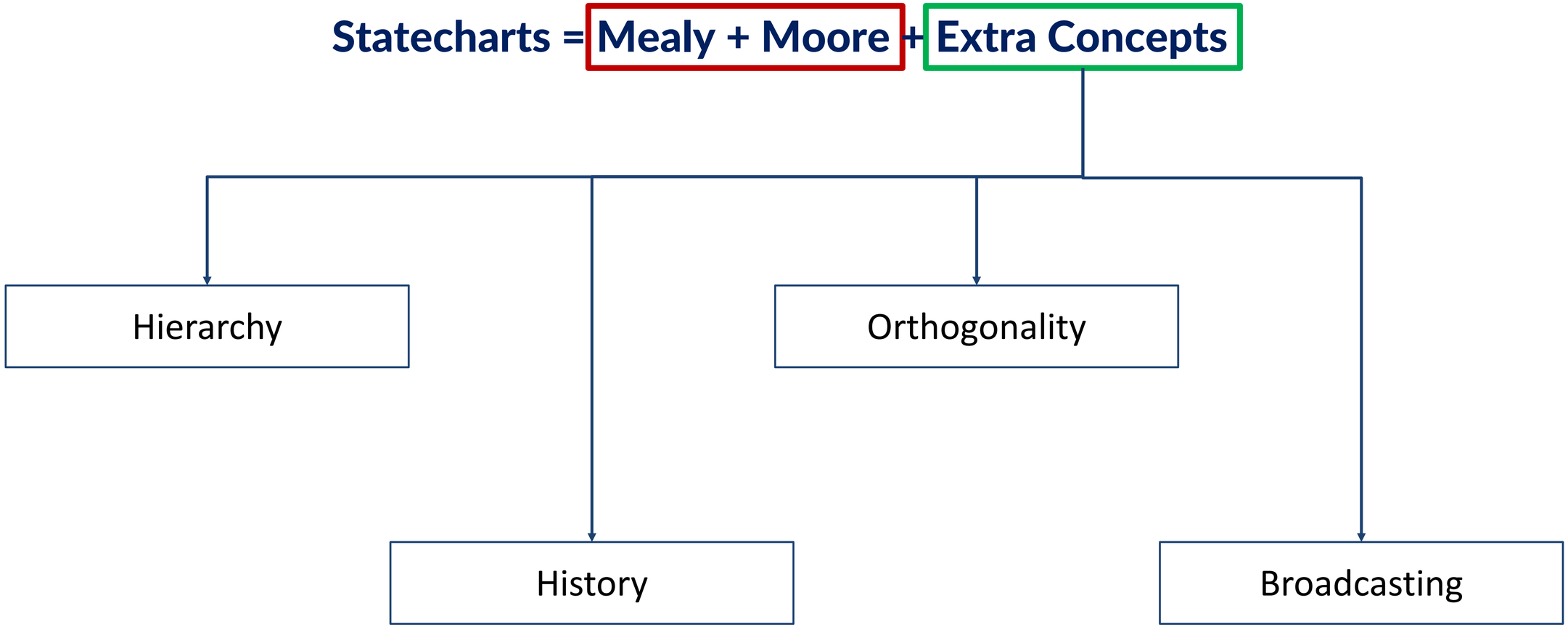
The History:

Example: Let's assume that the motion system is connected sequentially with power off state and we are adding a **deep history**. Assuming last active was Gear_3 on Forward.

This means that when you power off then back on, you will go to **the last active** state in the **level of the Car Motion.** That means you will go to the last active state in Forward {Gear_3}



Statecharts



Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Orthogonality:

- In Statecharts, orthogonality refers to the ability to ***concurrently*** execute multiple **independent** aspects or behaviors of a system.
- This concept allows for the modeling of complex systems in a modular and manageable way by representing different facets of the system as orthogonal components that **can progress independently of each other**.

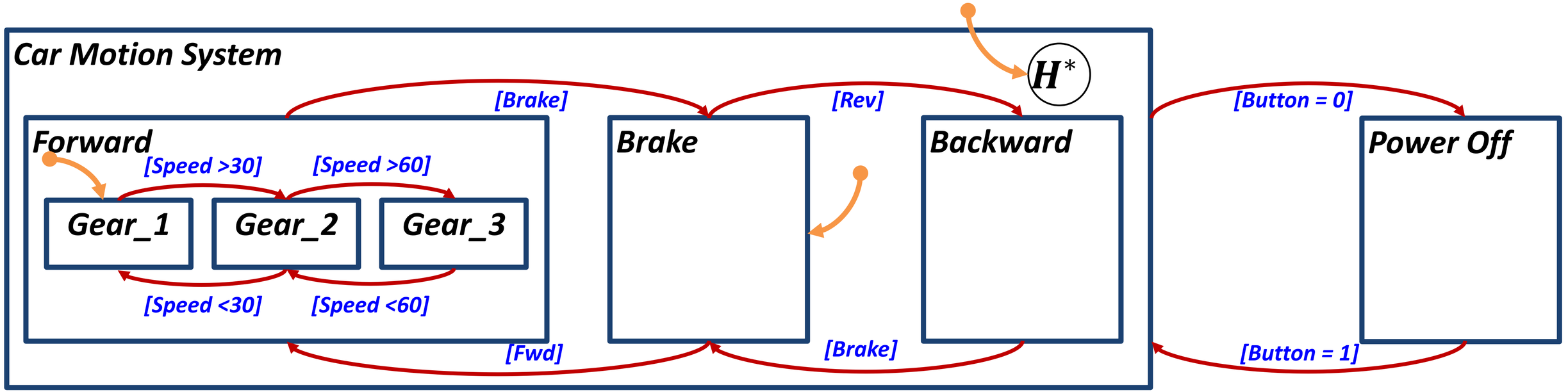
- The main idea behind orthogonality is to break down the overall behavior of a system into independent and concurrently executing parts, each represented by a separate set of states and transitions.
- This is achieved through the use of concurrent states.

Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Orthogonality:

Example: Let's add another subsystem beside the motion which is the window wiper. This subsystem is independent on the motion.

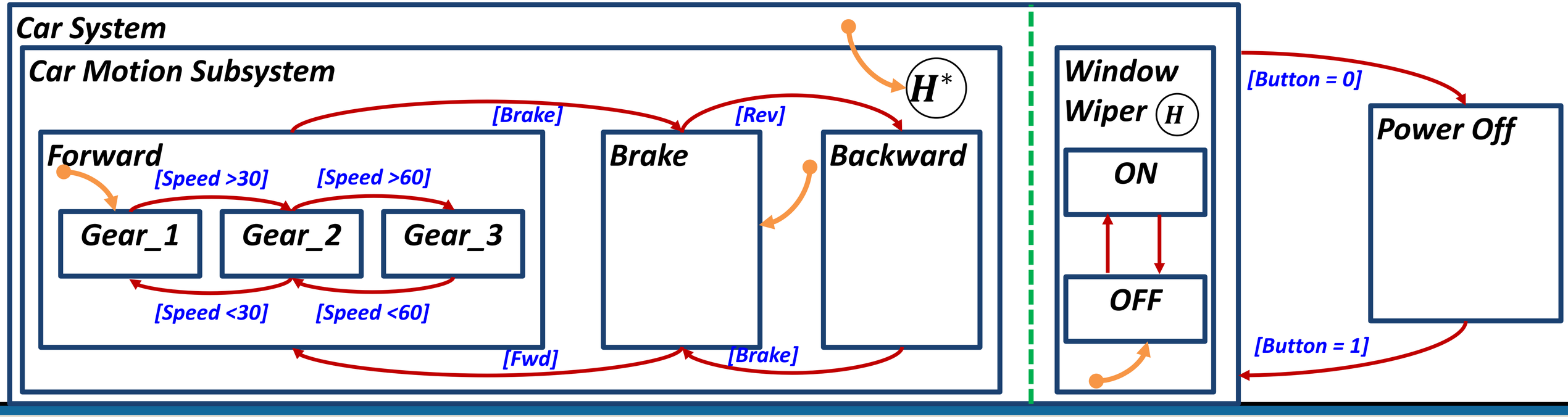


Statecharts = Mealy + Moore + Extra Concepts

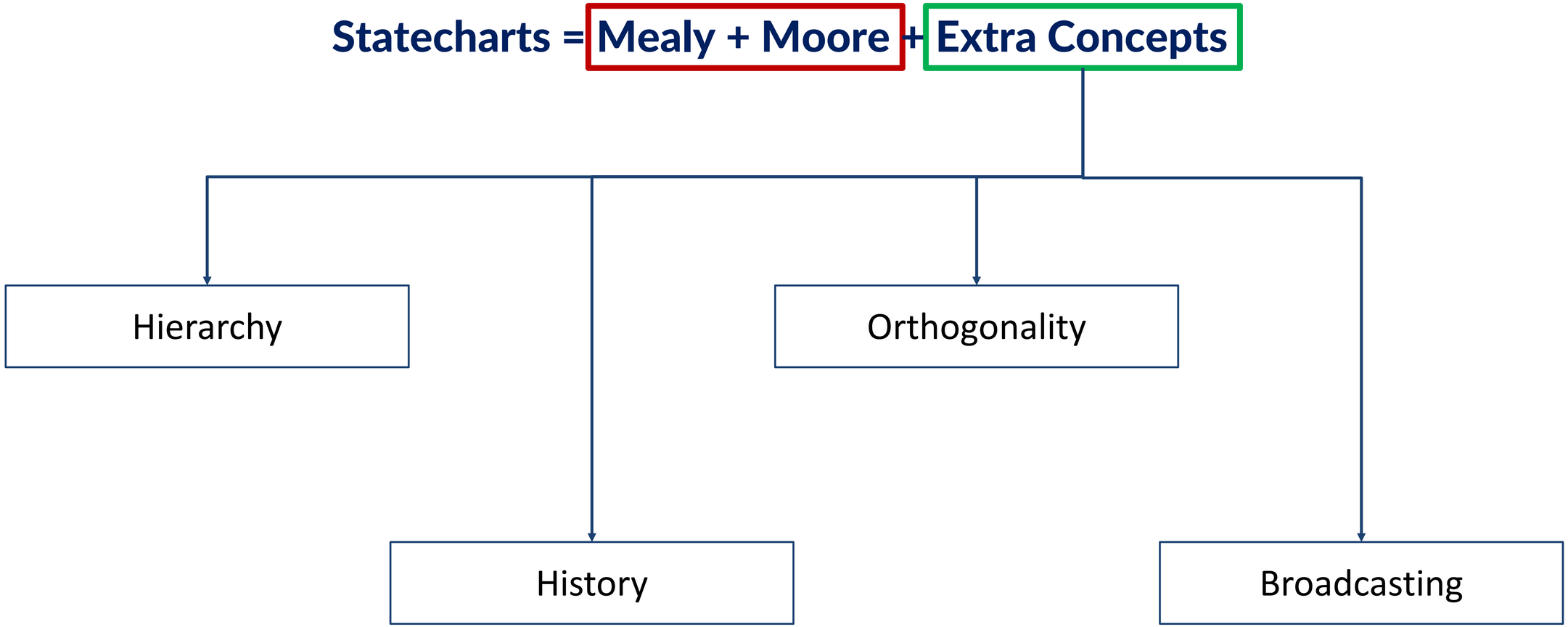
The Orthogonality:

Example: Which means that it should be concurrently modeled. So we add the new with the substates {On, OFF}.

To say that both are running concurrently, we add a ***dashed line*** between them.



Statecharts



Statecharts

Statecharts = Mealy + Moore + Extra Concepts

The Broadcasting:

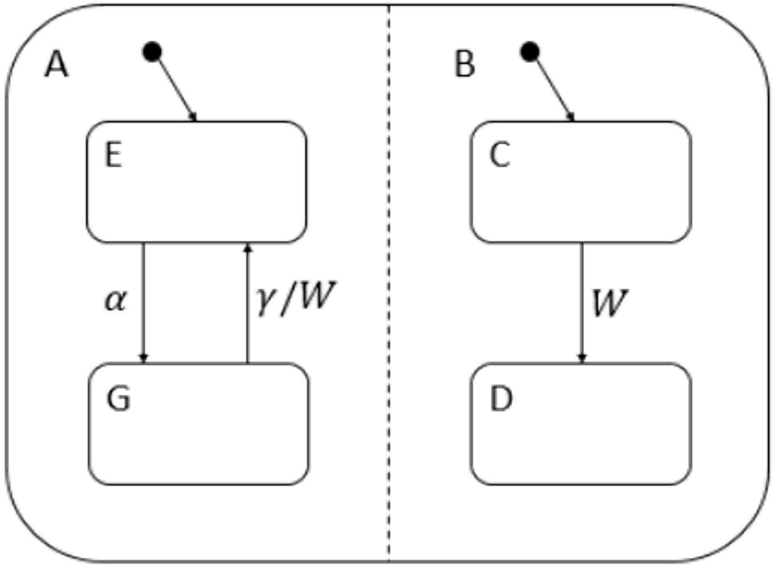
- In the context of Statecharts, broadcasting refers to a mechanism where an **event triggered in one part** of the system is simultaneously delivered or **sent to multiple components or states** within the system.
- This concept is particularly useful in scenarios where **multiple parts** of the system need to **respond to the same event independently**.
- Broadcasting events allows for a **form of communication** between different states or components that are not necessarily in a direct hierarchical relationship.
- Instead of relying on a specific target state to receive an event, the event is broadcasted to multiple potential recipients, and each recipient decides how to react to the event based on its own internal logic.

Statecharts = Mealy + Moore + Extra Concepts

The Broadcasting:

Example:

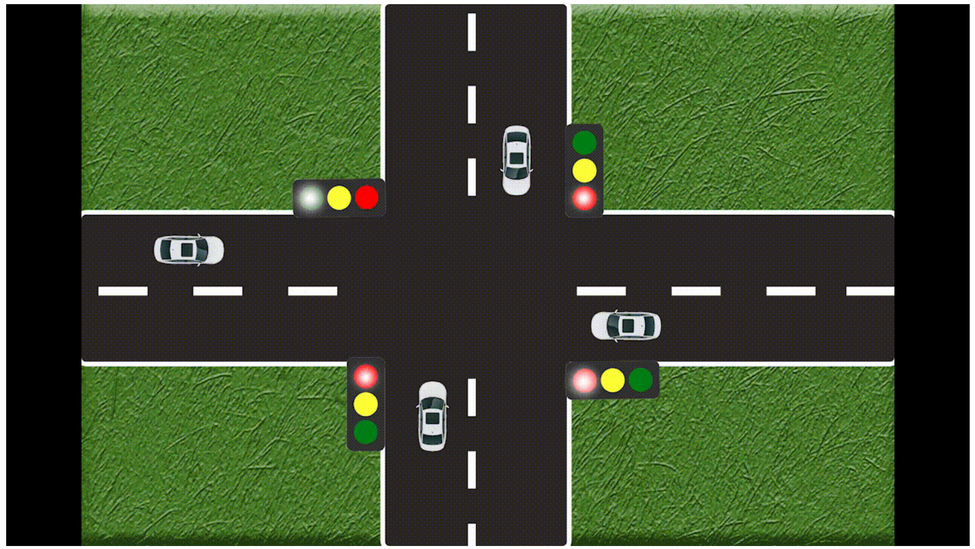
- If G is active and it is triggered with γ, then an action W is made in the part of the subsystem A.
- Due to the broadcasting if C is the active state in the part of the subsystem B, then the state will be triggered and state will change to D as a result to the W trigger.



Example

Traffic Light Controller: *Description*

- In this example, we will explore how Finite State Machines (FSMs) can be used to model a Traffic Light Controller. Traffic light systems are a ubiquitous part of our daily lives, and they provide an excellent context for understanding how FSMs can be applied to real-world embedded systems.
- Imagine a standard traffic intersection with traffic lights controlling the flow of vehicles and pedestrians. The system's goal is to manage the traffic efficiently and safely by providing the right-of-way to various directions of traffic. The traffic light controller's task is to switch between red, green, and yellow lights for each direction, following a specific sequence.



Example

Traffic Light Controller: *FSM Solution*

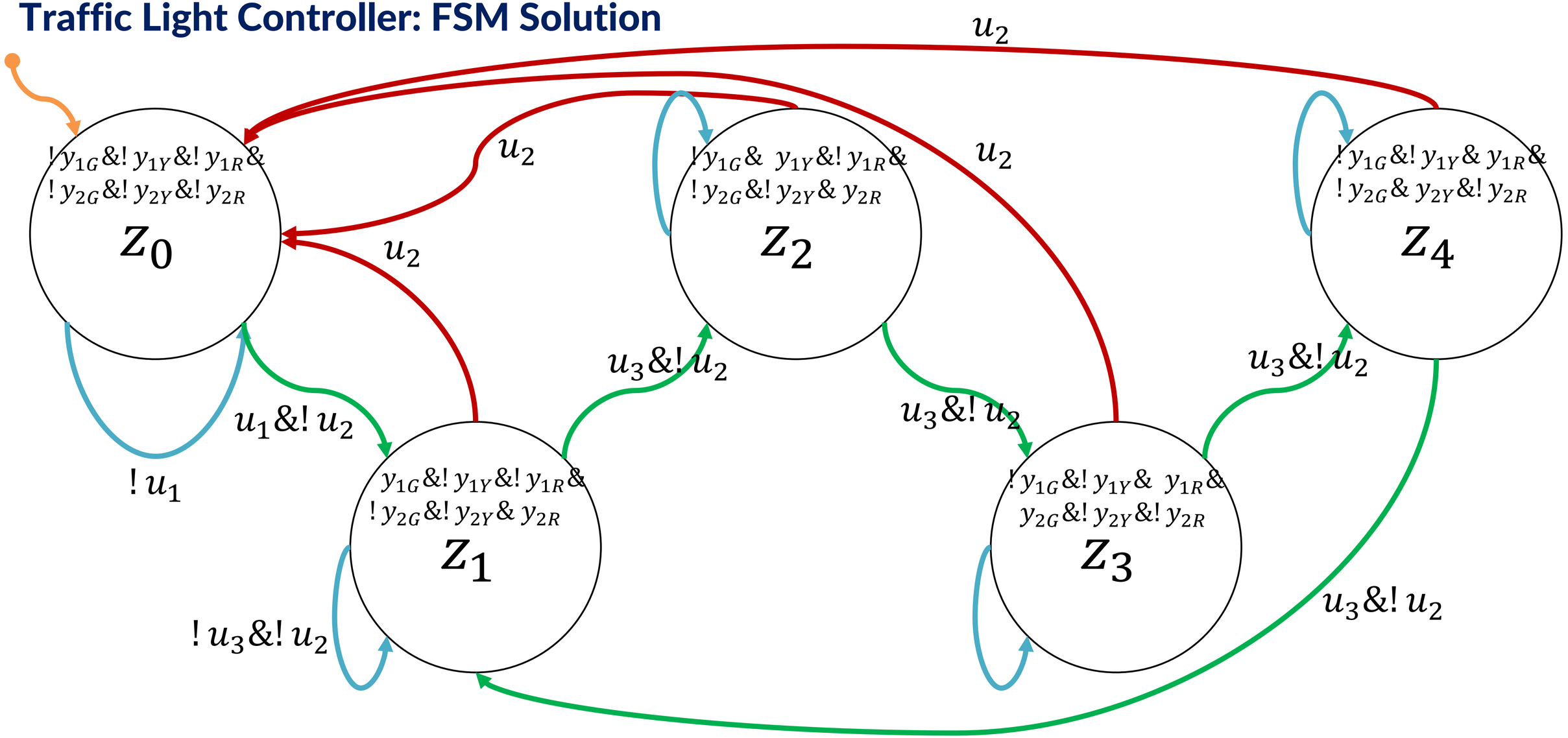
- **States:**
  1. Idle → z_0
  2. North-South Green, East-West Red → z_1
  3. North-South Yellow, East-West Red → z_2
  4. North-South Red, East-West Green → z_3
  5. North-South Red, East-West Yellow → z_4

- **Inputs:**
  1. Start Button → u_1
  2. Reset Button → u_2
  3. Timer → u_3

- **Outputs:**
  1. 3 RGY lambs for North-South traffic light → y_1, y_2, y_3
  2. 3 RGY lambs for , East-West traffic light → y_4, y_5, y_6

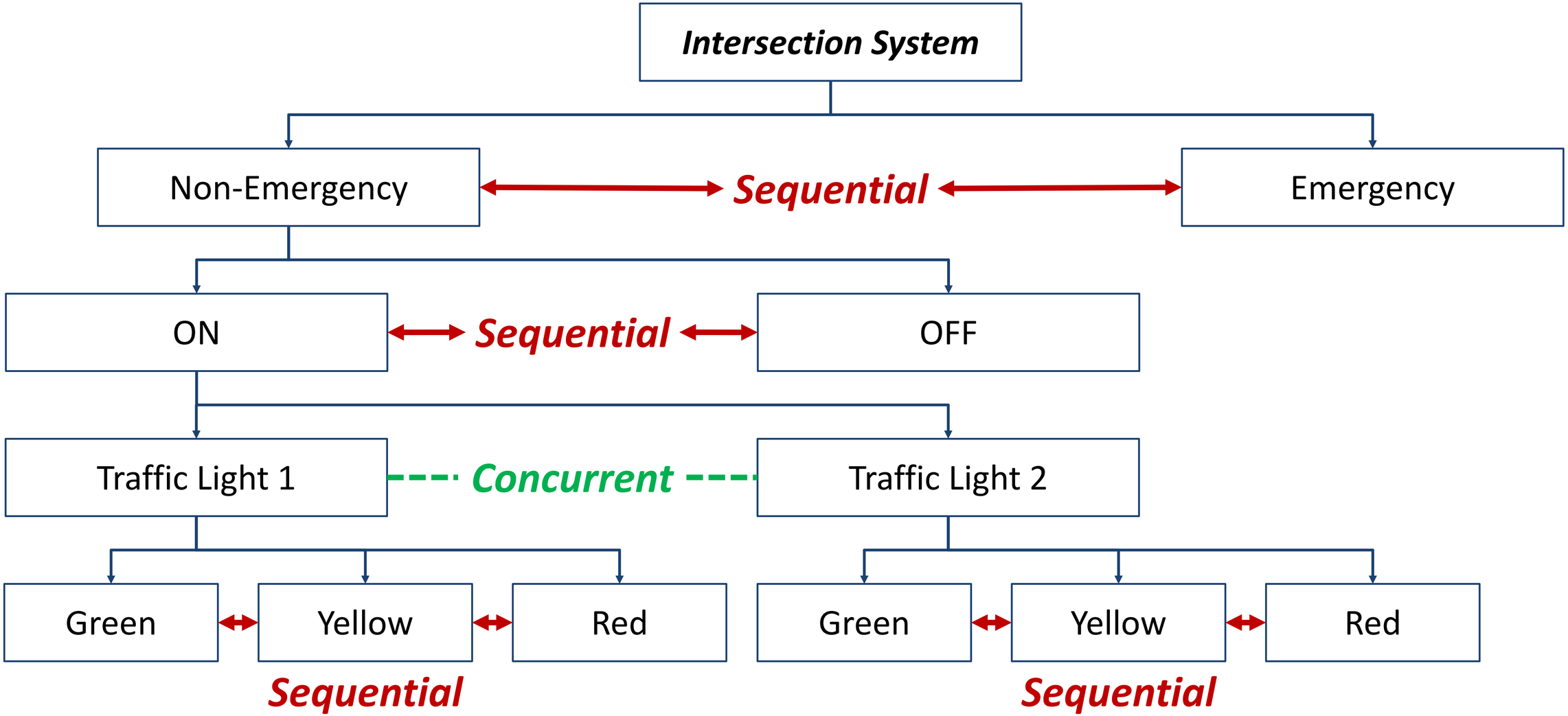
Example

Traffic Light Controller: FSM Solution

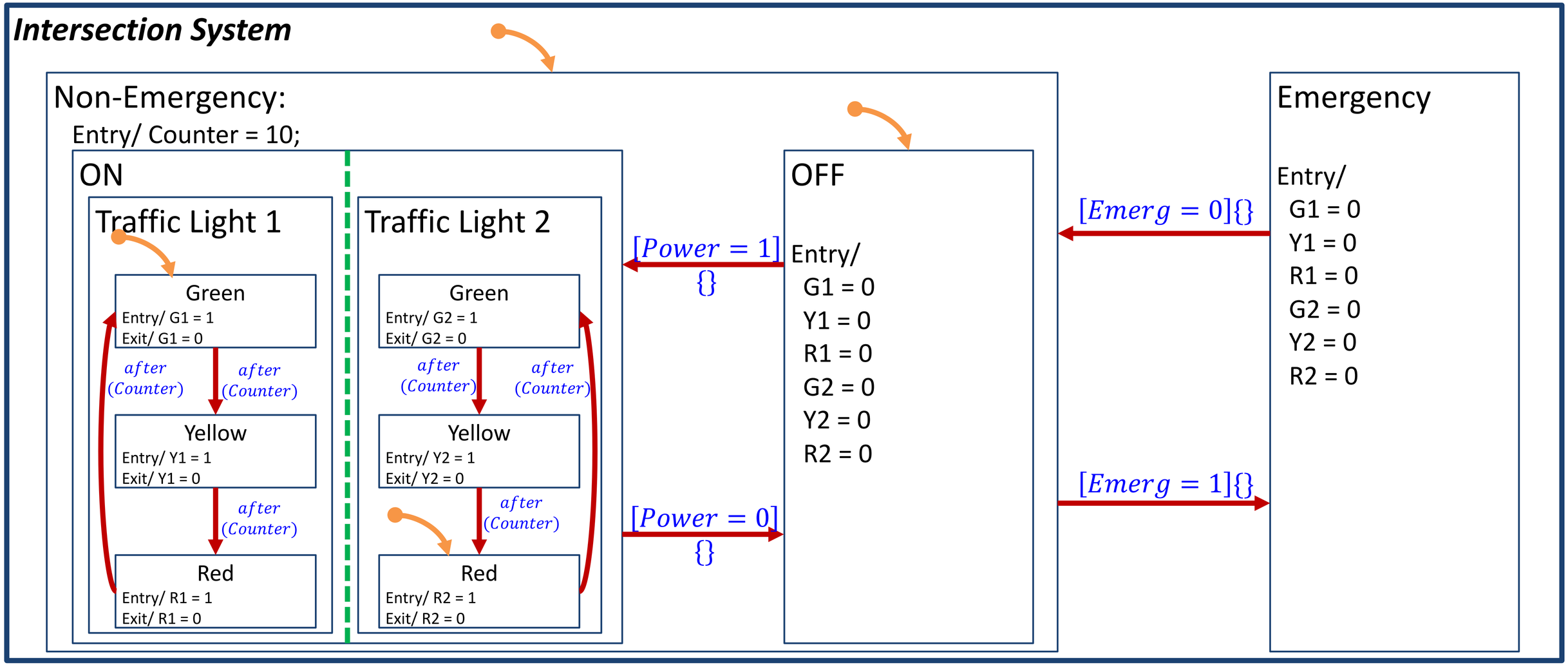


Example

Traffic Light Controller: *SC Solution*

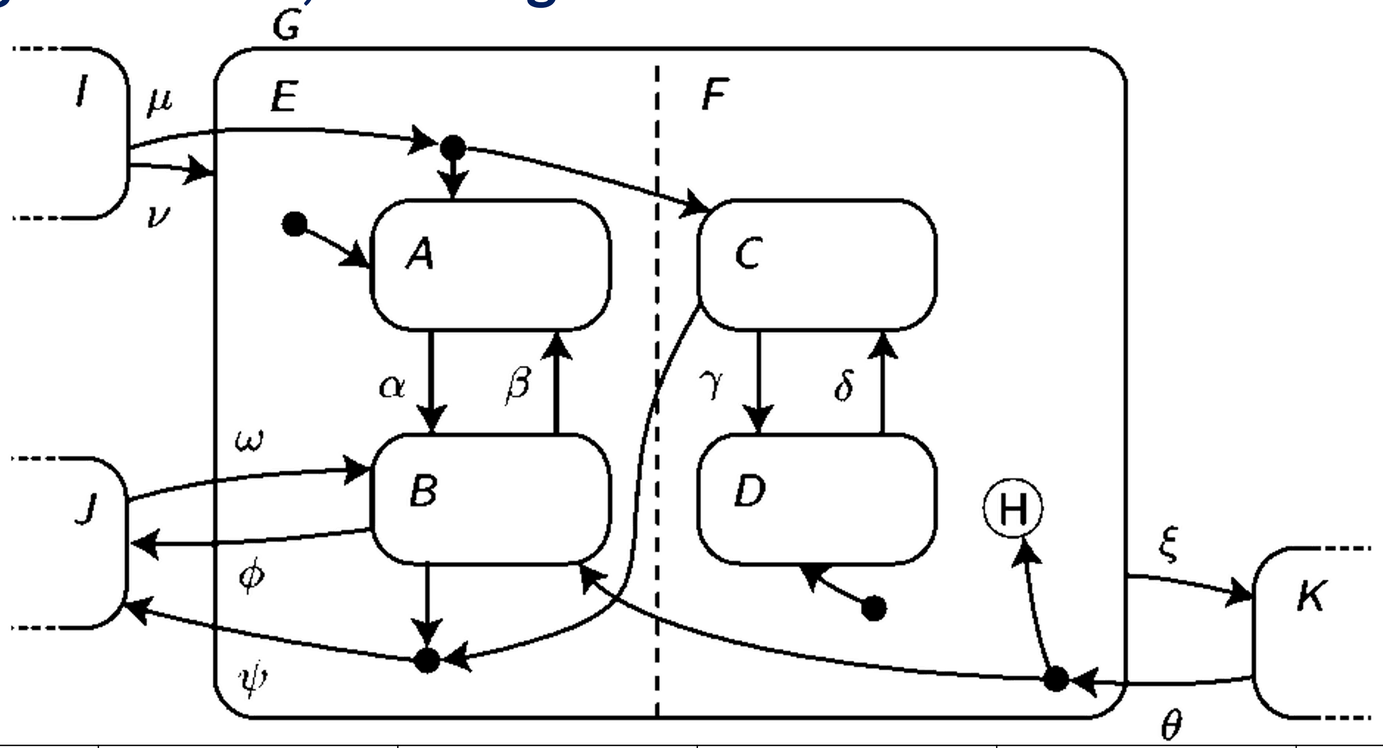


Traffic Light Controller: SC Solution



Example

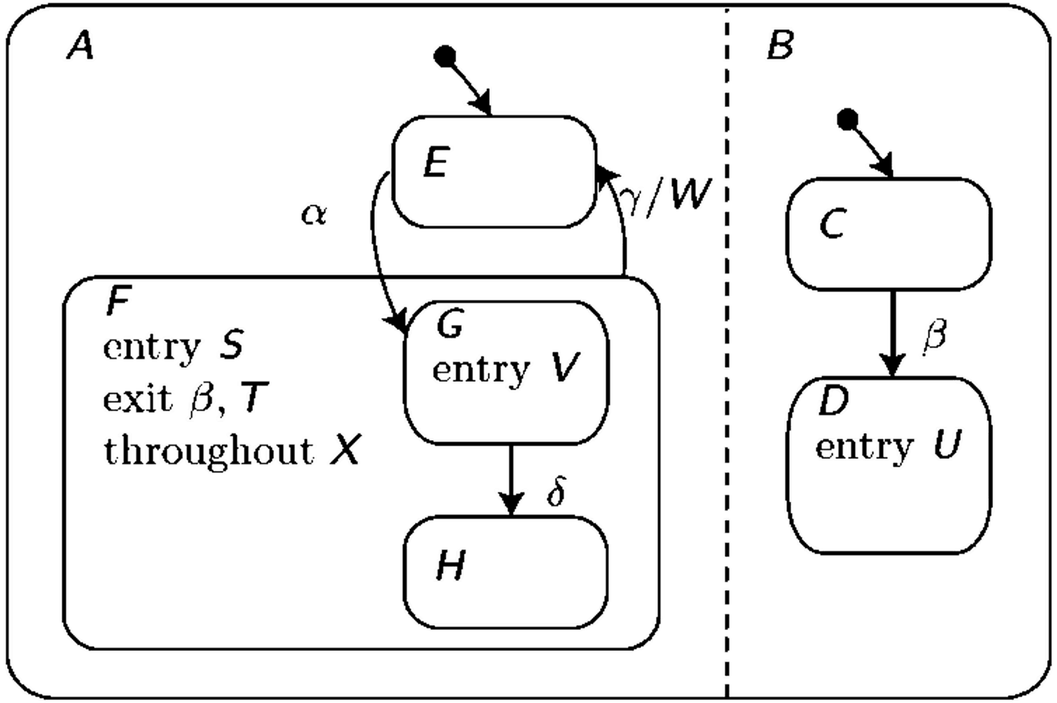
Trace the following Statechart, knowing that last active state in F is C



$z(k)$	I	I	J	K	$(B, C/D)$	B, C	$(A/B\ ,\ C/D)$
$u(k)$	μ	v	w	θ	ϕ	ψ	ζ
$z(k+1)$	A, C	A, D	B, D	B, C	J	J	K

Example

State the generated events/states of the statechart



$u(k)$	$-$	α	δ	γ	$-$
$z(k)$	E, C	G, C	H, C	E, C	E, D
$y(k)$	$-$	S, V, X	X	β, T, W	U

For Further Inquiries, Please send an email

Catherine.elias@guc.edu.eg,
Catherine.elias@ieee.org

Thank you for your attention!

See you next time ☺